[15] `n>>k` is normally interpreted as “n is much larger than k”.

[16] `O(log(k))` to be precise, but since `k` is a constant (much smaller than, and not depending on `n`), then `O(log(k))==O(1)`.

[17] Additional challenges can occur if the text is so large that either it can’t be contained in memory, or anyway a map-reduce approach is advisable: we encapsulate all that complexity in the `ComputeFrequencies` method.

[18] Instead of the actual frequency it might be easier, and equivalent for the algorithm, just counting the number of occurrences of each character.

[19] It’s simply going to be a 1:1 mapping over the characters in the text, with extra attention needed to efficiently construct the bits encoding in output.

[20] This section is includes advanced concepts.

[21] The Fibonacci heap is an advanced version of priority queue that is implemented with a set of heaps. For a Fibonacci heap, find-minimum, insert and update priority operations take constant amortized time, `O(1)`, while deleting an element (including the minimum) requires `O(log n)` amortized time, where `n` is the size of the heap.
So, in theory, Fibonacci heaps are faster than any other heap, although in practice, being overly complicated, their implementations end up being slower than simple binary heaps’.

[22] http://comjnl.oxfordjournals.org/content/34/5/428

[23] `O(n*log(n))`, for an input of size `n`.

[24] Where `D` is the branching factor for the d-ary heap under analysis.

[25] It is, obviously, possible to find a formula for `f(D)`’s minima using calculus, and in particular computing the first and second order derivatives of the function.

[26] We used copyright-free novels downloaded from Project Gutenberg, http://www.gutenberg.org (worth checking out!)

[27] Disclaimer: there are many possible ways to do this, and possibly better ways too. This is just one of the possible ways.

[28] In out Python implementation, the names become respectively `_push_down` and `_highest_priority_child`, to follow Python naming conventions.